Session 1 - Getting Acquainted with GPU Programming
Geavanceerde computerarchitectuur: labosessie

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

February 12, 2026



Year: 2026

Class: MELEC

Instructor:

Contents

1 Context **2**

2 Flip array **2**
2.1 Methodology 2
2.2 Observations/results 2

3 Conclusion **3**

A Code **3**

1 Context

Het doel van deze eerste labosessie is om vertrouwd te raken met de basisprincipes van GPU-programmering met behulp van CUDA. Hierbij is het belangrijk om de relatie tussen de GPU-architectuur (cores) en de programmeermodellen (grids, blocks en threads) te begrijpen.
De opdracht om een programma te schrijven dat een array van integers "flipt"[2].

2 Flip array

2.1 Methodology

We gebruiken CUDA om de GPU te programmeren, dit heeft als voordeeel dat we secuentiel kunnen werken (dit is het doel van deze labo's). In plaats van een sequentiële lus op de CPU, definiëren we een kernel functie die N keer parallel wordt uitgevoerd door N verschillende CUDA-threads.
De werking van de code kan worden opgedeeld in drie stappen:

a. **Memory Allocation Migration:** We alloceren geheugen op de GPU en kopiëren de input-array $[0, 1, ..., 127]$.

b. **Kernel Execution:** We lanceren de kernel met een grid van thread blocks. Omdat we meerdere blocks met meerdere threads moeten gebruiken, is het cruciaal om de globale index van elke thread correct te berekenen. De globale index i wordt berekend als:

$$i = B_{lock}I_d \cdot B_{lock}D_{im} + T_{hread}I_d \tag{1}$$

Hiermee vinden we welk element van de array door welke thread wordt verwerkt. De thread leest waarde op positie i en schrijft deze naar positie $N - 1 - i$ in de output-array.

What is the largest possible block size (max. threads per block): De maximale block size is 1024 [1].

c. **Verification:** Het resultaat wordt teruggekopieerd naar het systeem geheugen om te controleren of de output daadwerkelijk $[127, ..., 1, 0]$ is.

2.2 Observations/results

Op basis van de output kunnen we zien dat het programma correct werkt. De array wordt zoals verwacht omgedraaid.

```
[Running] cd "/home/eliasneels/.gemini/antigravity/Geav. Comp. Architectuur/Labo 1/" && nvcc flip
cu -o flip && "/home/eliasneels/.gemini/antigravity/Geav. Comp. Architectuur/Labo 1/"flip
Input: 0 1 2 3 4 5 6 7 8 9 ...
Block Per Grid: 4
Output: 127 126 125 124 123 122 121 120 119 118 ...
Success! Array flipped correctly.

[Done] exited with code=0 in 3.768 seconds
```

Figure 1: Output

3 Conclusion

We leerden de basis van GPU-computing. Deze kennis evalueerden we aan de hand van de "Flip array" oefening.

A Code

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define N 128
#define BLOCK_SIZE 32

__global__ void flipKernel(int *d_in, int *d_out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        d_out[n - 1 - idx] = d_in[idx];
    }
}

int main() {
    int *h_in, *h_out;
    int *d_in, *d_out;
    size_t size = N * sizeof(int);

    // Allocate host memory
    h_in = (int *)malloc(size);
    h_out = (int *)malloc(size);

    // Initialize host array
    printf( Input:  );
    for (int i = 0; i < N; i++) {
        h_in[i] = i;
        if (i < 10) printf( %d  , h_in[i]);
    }
    printf( ...\n );

    // Allocate device memory
    cudaMalloc((void **)&d_in, size);
    cudaMalloc((void **)&d_out, size);

    // Copy input data to device
    cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice);

    // Launch kernel
    int threadsPerBlock = BLOCK_SIZE;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    printf( Block Per Grid: %d\n , blocksPerGrid);
    flipKernel<<<blocksPerGrid, threadsPerBlock>>>(d_in, d_out, N);

    // Copy result back to host
    cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost);

    // Verify result
    printf( Output:  );
    bool success = true;
    for (int i = 0; i < N; i++) {
        if (i < 10) printf( %d  , h_out[i]);
```

```
53          if (h_out[i] != N - 1 - i) {
54              success = false;
55          }
56      }
57      printf( ...\n );
58
59      if (success) {
60          printf( Success! Array flipped correctly.\n );
61      } else {
62          printf( Error! Array flip failed.\n );
63      }
64
65      // Free memory
66      free(h_in);
67      free(h_out);
68      cudaFree(d_in);
69      cudaFree(d_out);
70
71      return 0;
72  }
```

Listing 1: CUDA Kernel for Array Flipping

References

[1] Feys T. Gent (2026) *CUDA_C_Programming_Guide-exercise_1*, Lab notes.

[2] Feys T. Gent (2026) *session-1*, Lab notes.